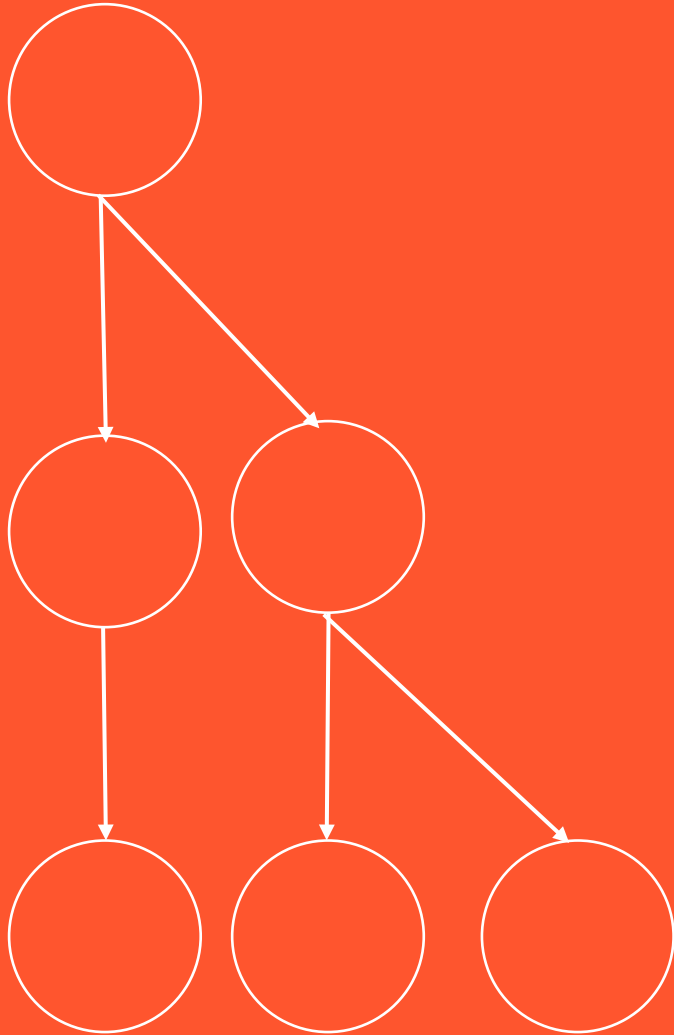


BCOG 100

Neural Circuits

Lecture 3: Logical Computation in Neural Circuits



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

Reading Check 5-3

Password: logic

Question: what's the most interesting thing you've learned in this class so far?



Logical Computation in Neural Circuits

Recap & Overview

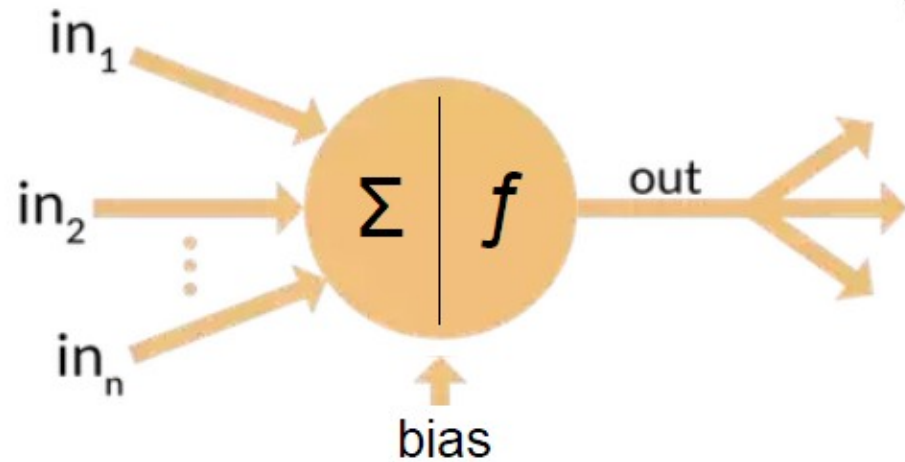
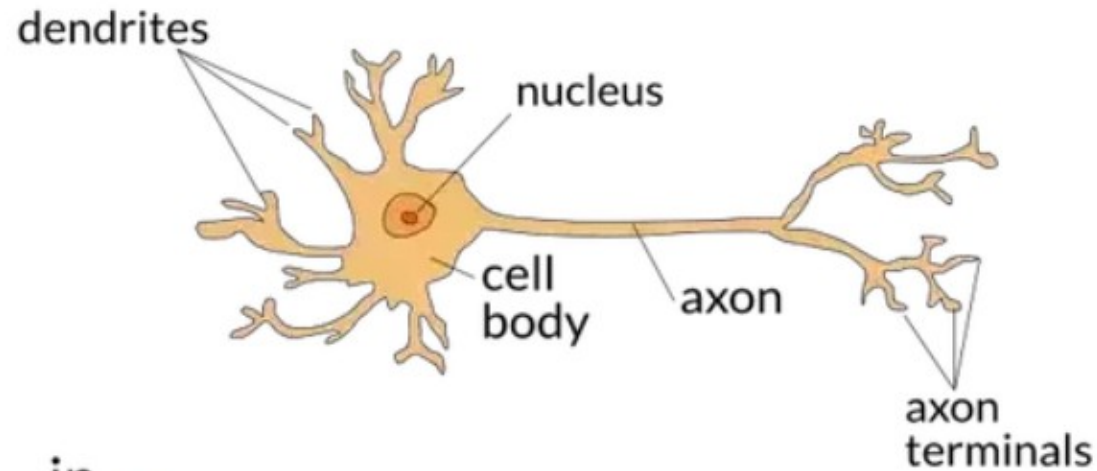
Lecture 1: Circuits as building blocks

Lecture 2: Circuits adapt & learn

Today: Circuits as computation



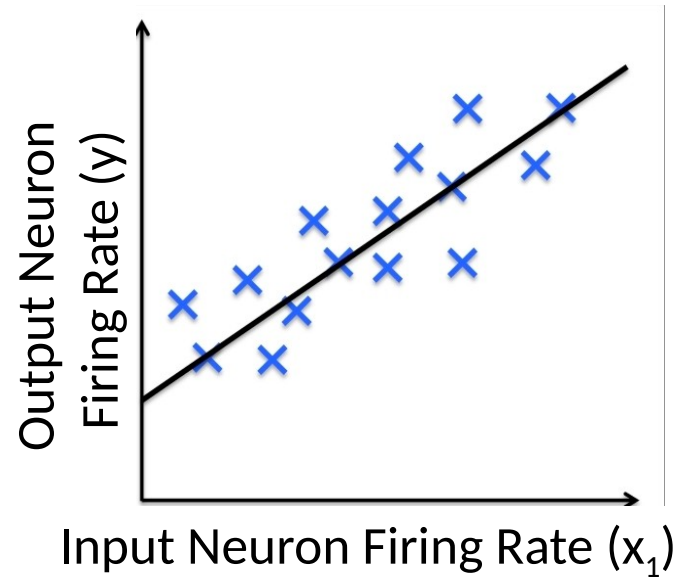
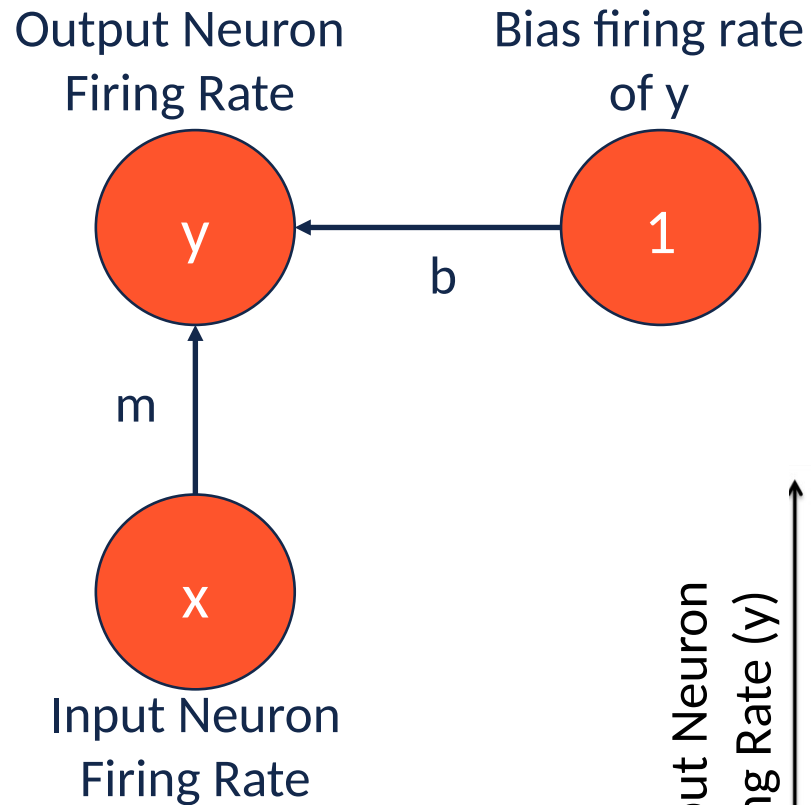
Logical Computation in Neural Circuits



From Biology to Abstraction

- Real neurons: graded, noisy, firing rates
- Models: simplified on/off threshold units
- Abstraction = useful but limited

Logical Computation in Neural Circuits



Formal Model of Neurons

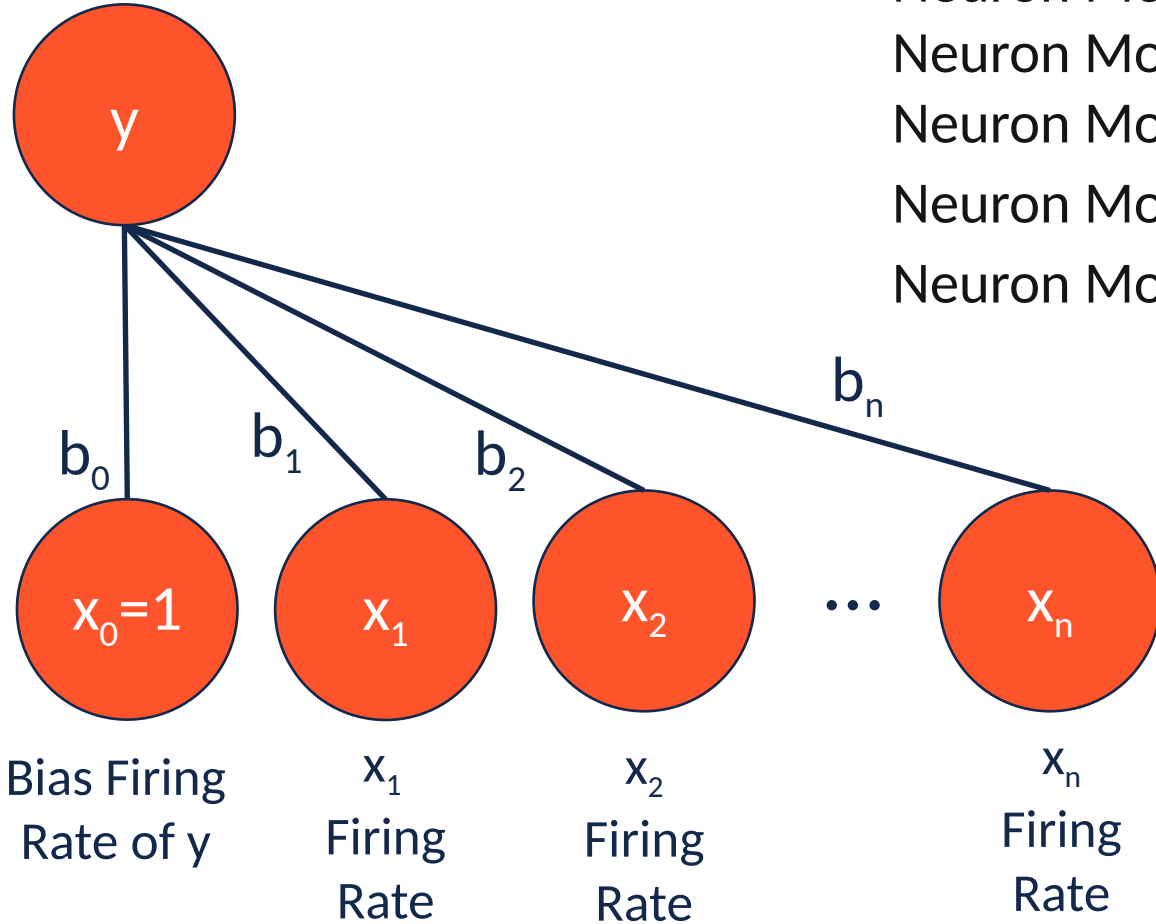
Neuron Model: $y = m * x + b$

- y : output neuron firing rate
- x : input neuron firing rate
- m : effect x (input neuron) has on y (output neuron)
- b : base firing rate of y when x is 0

Neuron Model: $y = m * x + b * 1$

Logical Computation in Neural Circuits

Output Neuron
Firing Rate



Expanded Formal Model of Neurons

Neuron Model: $y = m * x + b * 1$

Neuron Model: $y = b * 1 + m * x$

Neuron Model: $y = b * x_0 + m * x_1$, where $x_0 = 1$

Neuron Model: $y = b_0 * x_0 + b_1 * x_1$, where $x_0 = 1$

Neuron Model: $y = b_0 * x_0 + b_1 * x_1 + b_2 * x_2 + \dots + b_n * x_n$, where $x_0 = 1$

$$y = \sum_{i=0}^n b_i * x_i, \text{ where } x_0 = 1$$

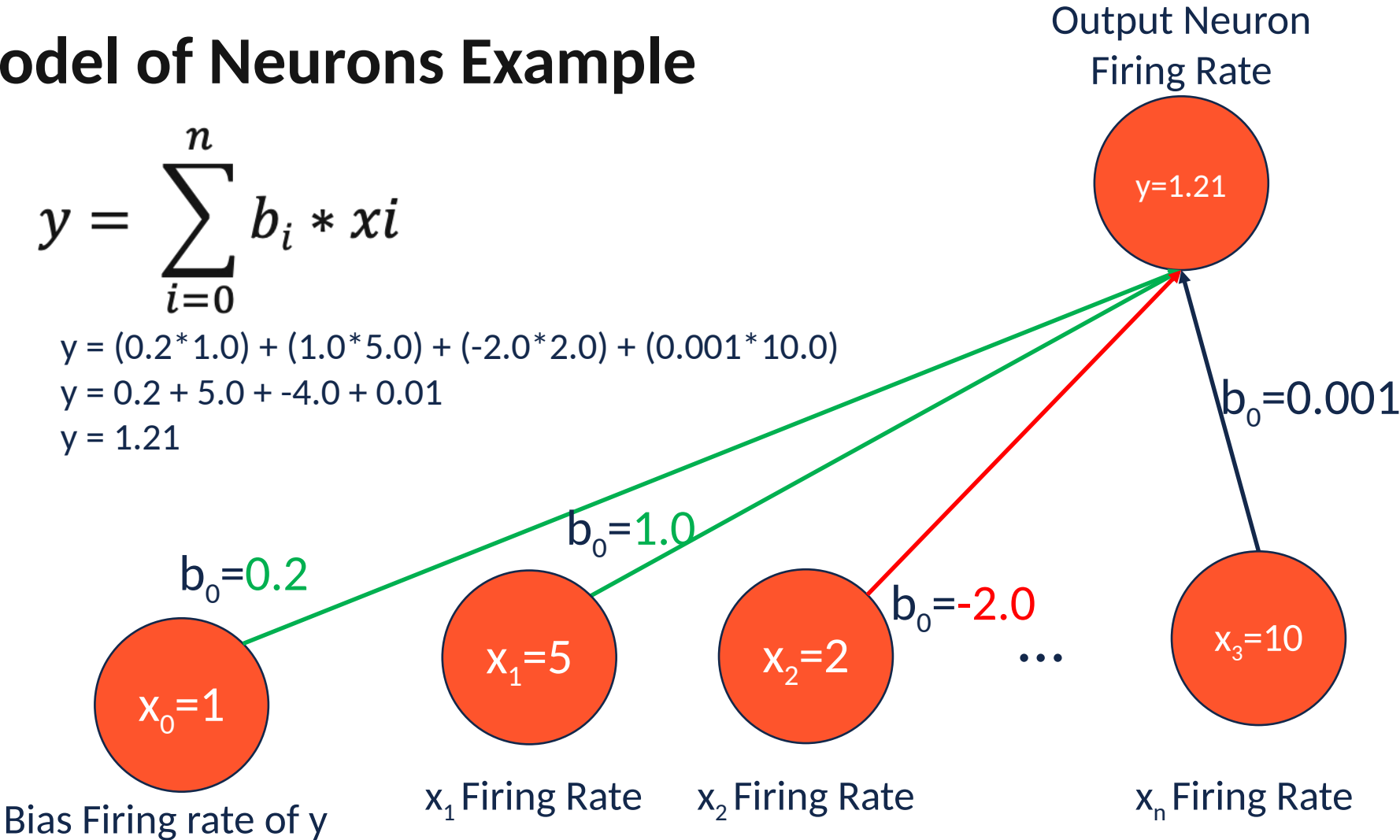
Logical Computation in Neural Circuits

Expanded Formal Model of Neurons Example

Neuron	b	x	x*b
x ₀ (bias)	0.2	1.0	0.2
x ₁	1.0	5.0	5.0
x ₂	-2.0	2.0	-4.0
x ₃	0.001	10.0	0.01
Net input into y			1.21

$$y = \sum_{i=0}^n b_i * x_i$$

$y = (0.2 * 1.0) + (1.0 * 5.0) + (-2.0 * 2.0) + (0.001 * 10.0)$
 $y = 0.2 + 5.0 + -4.0 + 0.01$
 $y = 1.21$

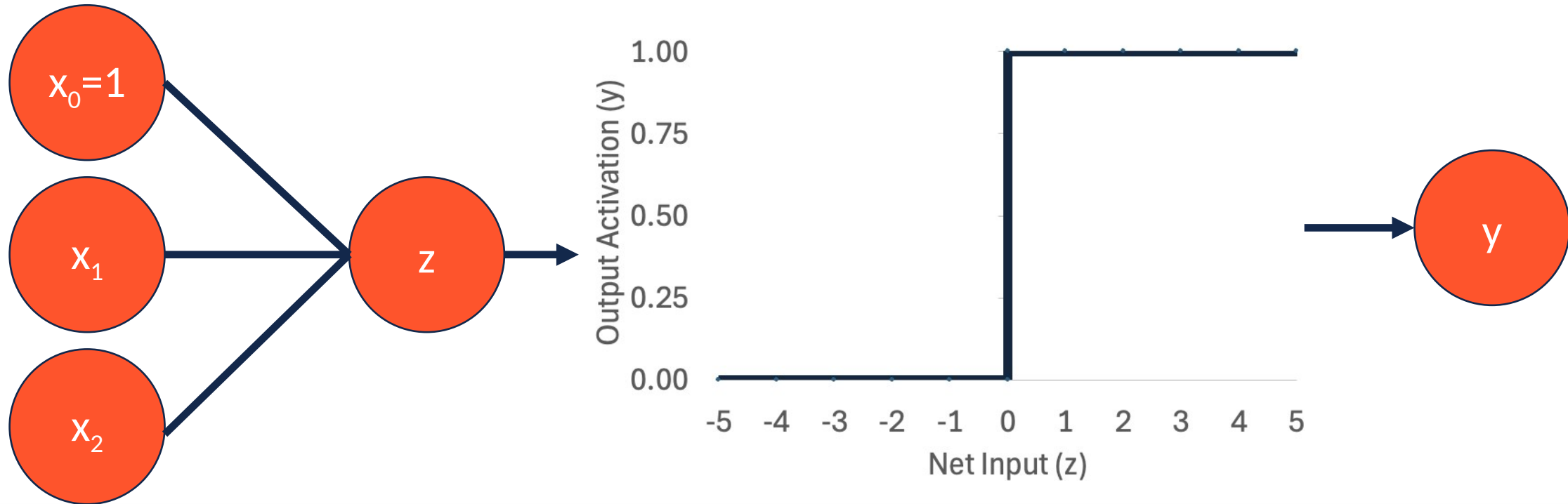


Logical Computation in Neural Circuits

Interpreting Output

Option 1: Step function → on/off, logic gates

- Net Input: $z = \sum_{i=0}^n b_i * x_i$
- Binary Threshold Activation Function Output: $y = f(z)$,
- where $f(z) = 1$ where $z \geq 0$, 0 otherwise

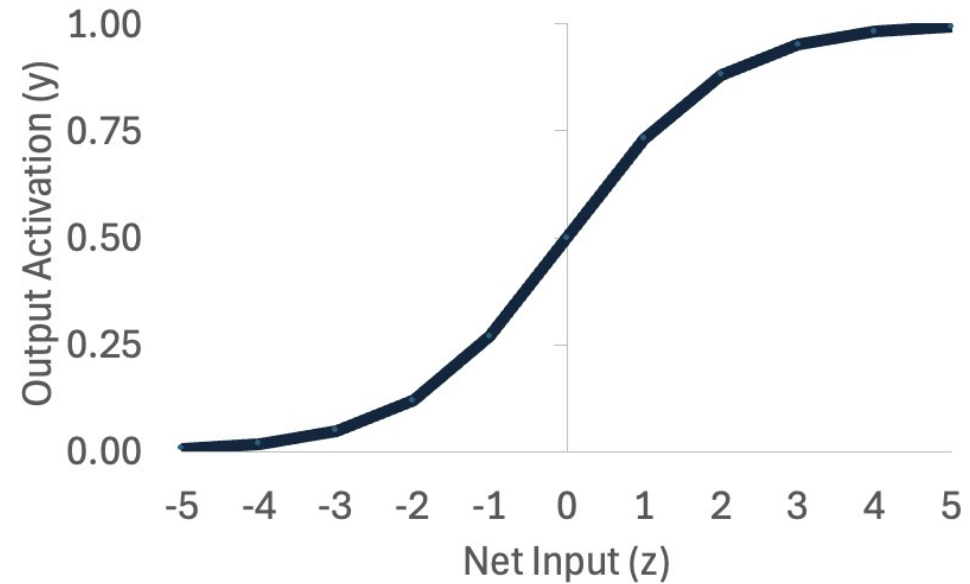
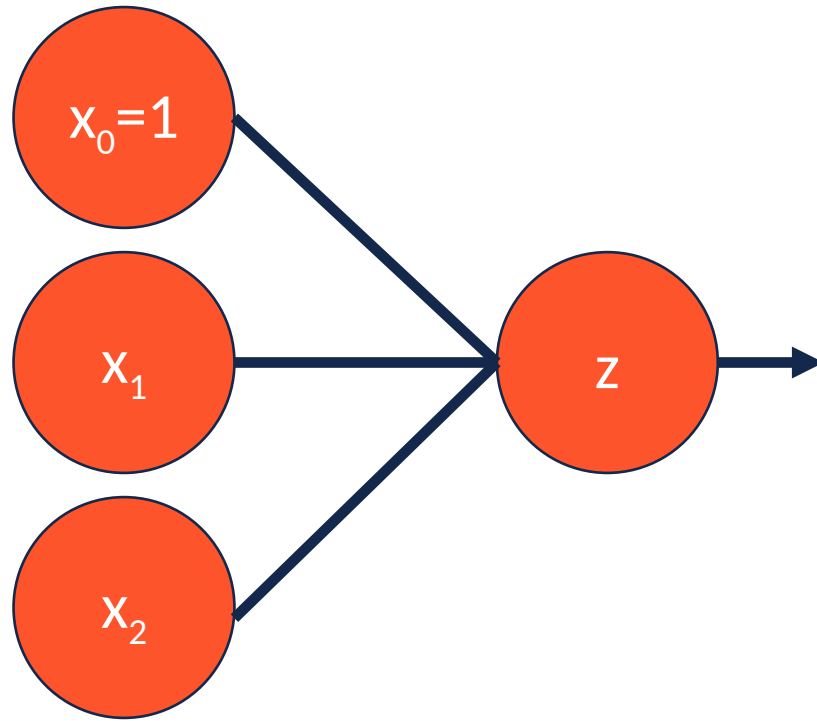


Logical Computation in Neural Circuits

Interpreting Output

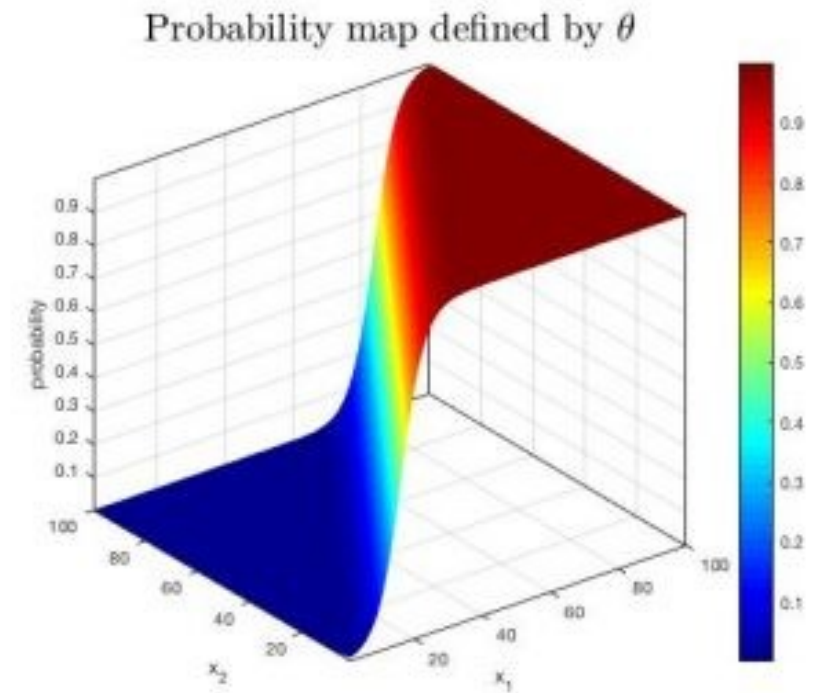
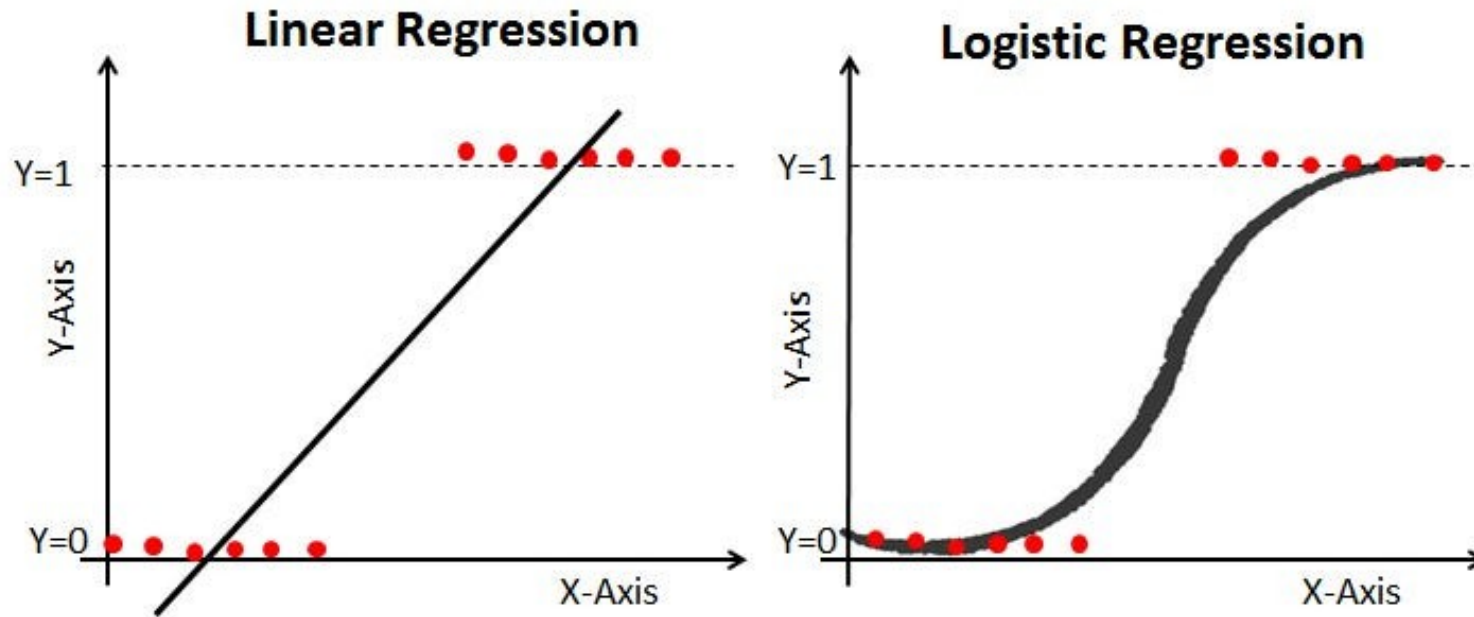
Option 2: Continuous Firing Rates

- Net Input: $z = \sum_{i=0}^n b_i * x_i$
- Sigmoid Function Output:
 $y = f(z)$, where $f(z) = 1 / (1 + e^{-z})$



Logical Computation in Neural Circuits

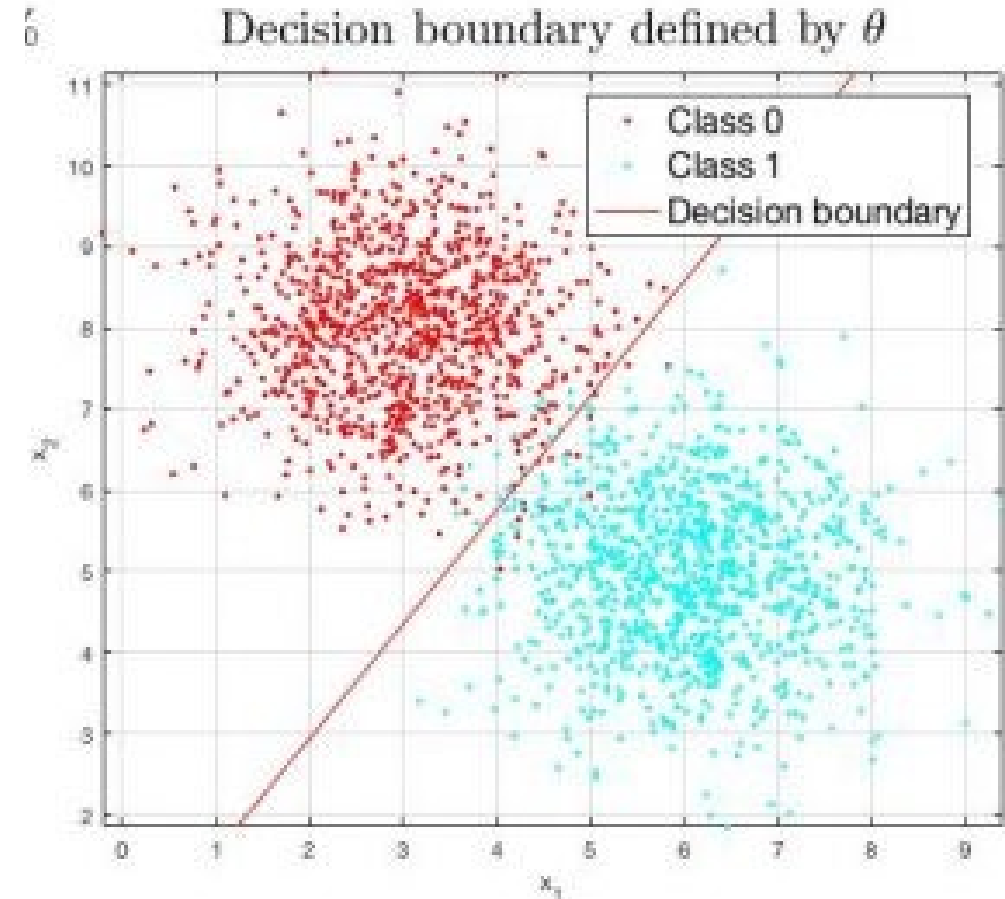
Logistic Regression and Classifier Models



Logical Computation in Neural Circuits

Logistic Regression and Classifier Models

- Learning a decision boundary between input classes
- On one side of the boundary, output neuron should fire at its maximum rate
- On the other side of the boundary, output neuron should not fire at all



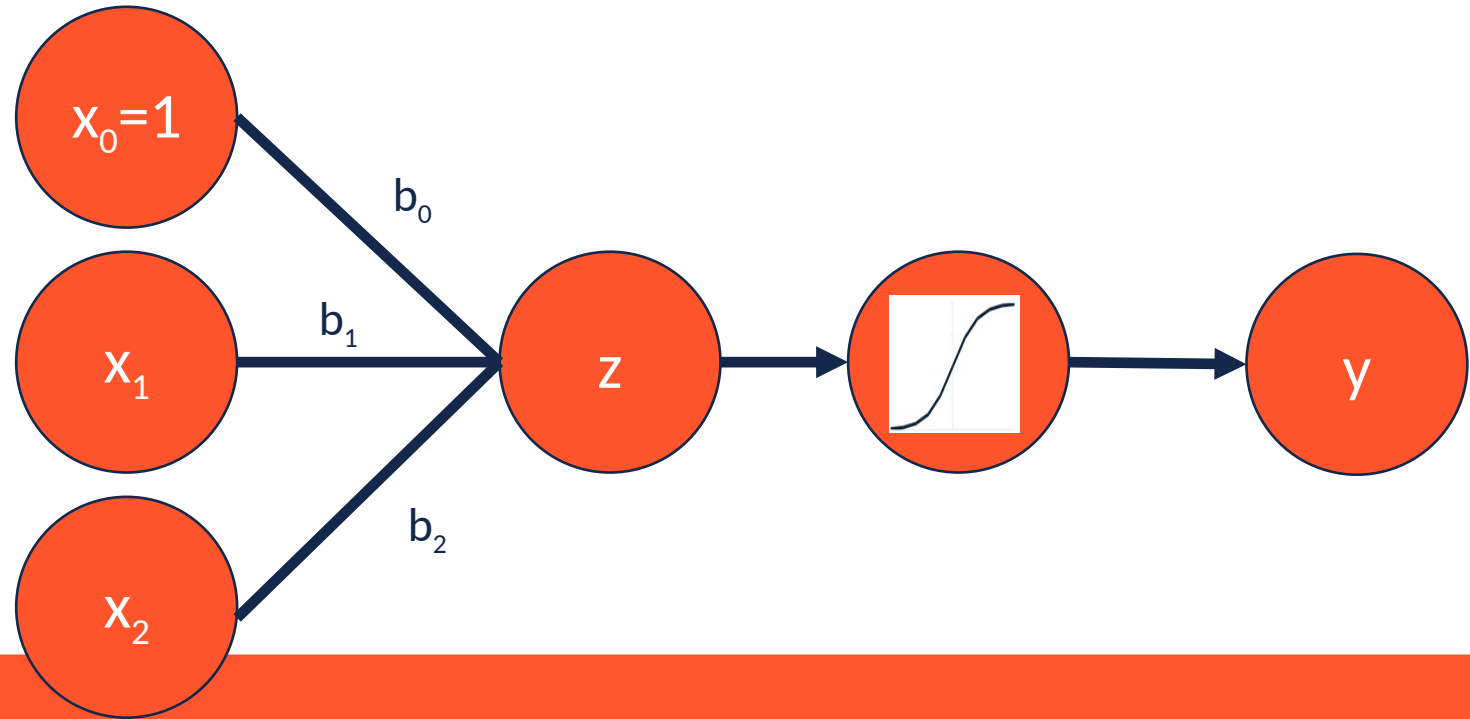
Logical Computation in Neural Circuits

Logic as Target Functions

- Boolean functions = desired input-output patterns
- AND, OR, NOT → truth tables
- Our goal: approximate these with sigmoid neurons
- Is there a set of weights (b_0 , b_1 , and b_2) that can output something like the value of y for every combination of x_1 and x_2 , such that this circuit calculates OR over its inputs?

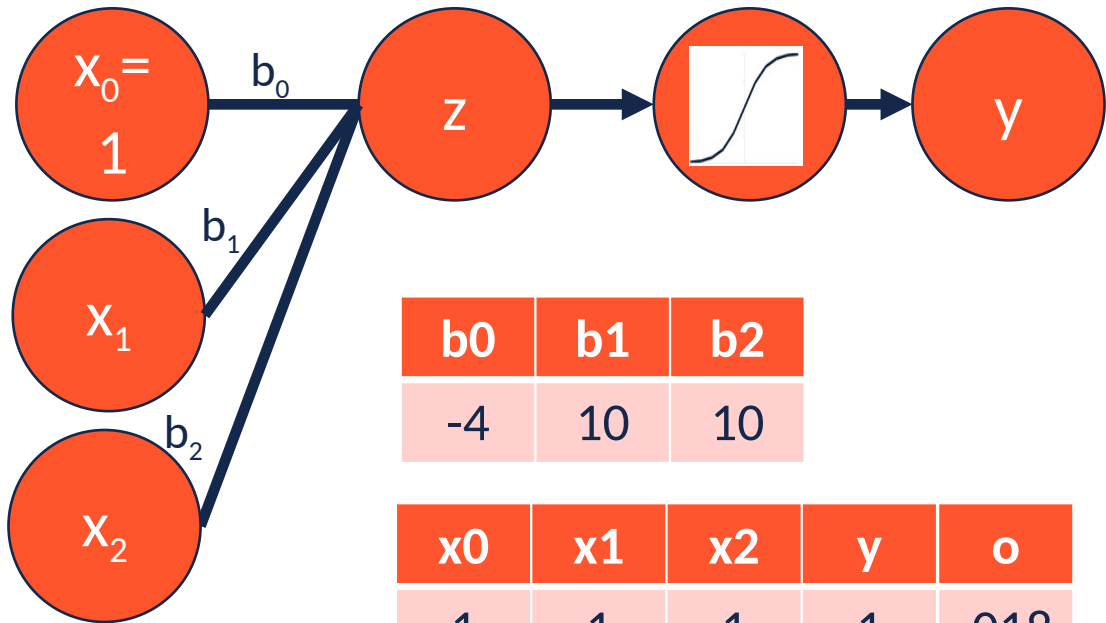
Truth Table for OR

x_1	x_2	y
1	1	1
1	0	1
0	1	1
0	0	0



Logical Computation in Neural Circuits

Logical OR Neural Circuit



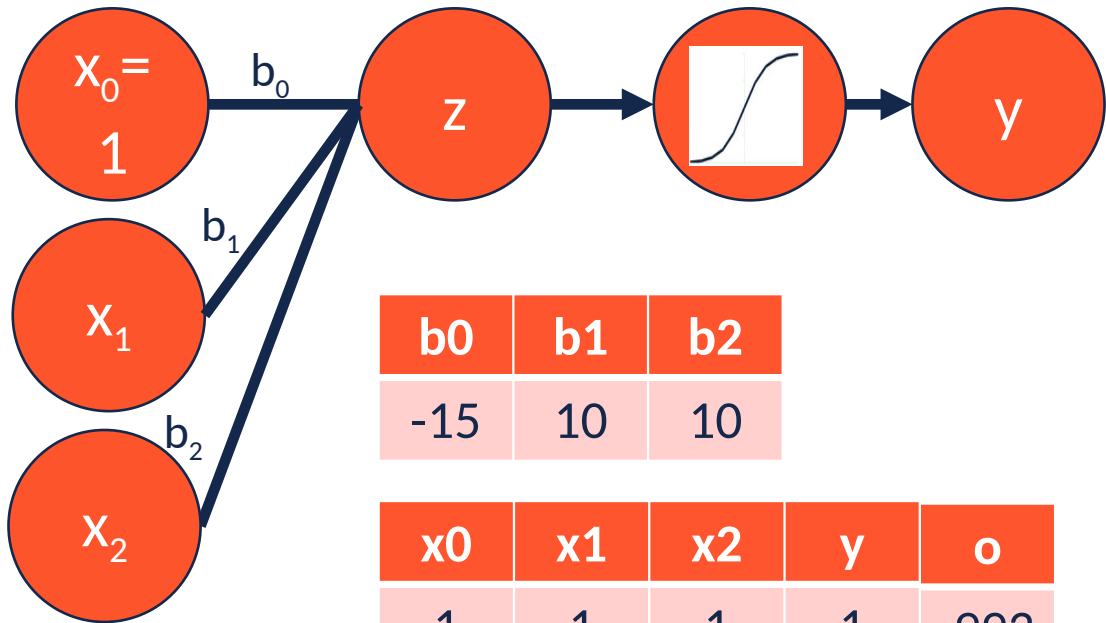
b_0	b_1	b_2
-4	10	10

x_0	x_1	x_2	y	o
1	1	1	1	.018
1	1	0	1	.998
1	0	1	1	.998
1	0	0	0	.999

Neuron	b	x	$x*b$
x_0	-4	1	-4
x_1	10	0	0
x_2	10	0	0
z			-4
$y = \text{sigmoid}(z)$			0.018

Logical Computation in Neural Circuits

Logical AND Neural Circuit



b_0	b_1	b_2
-15	10	10

x_0	x_1	x_2	y	o
1	1	1	1	.993
1	1	0	0	.007
1	0	1	0	.007
1	0	0	0	.001

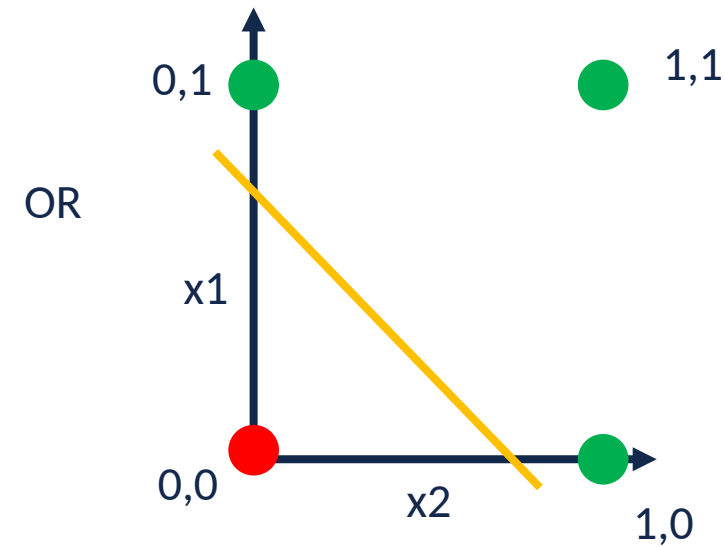
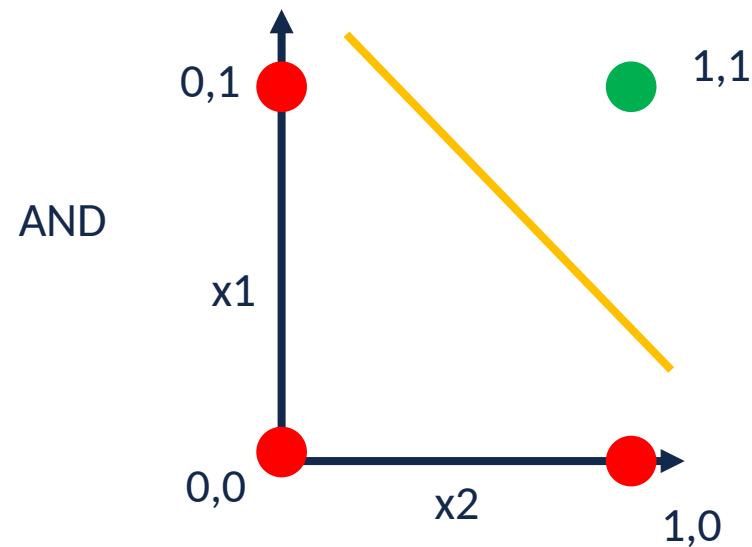
Neuron	b	x	$x*b$
x_0	-15	1	-15
x_1	10	0	0
x_2	10	0	0
z			-15
$y = \text{sigmoid}(z)$			<0.001

Logical Computation in Neural Circuits

Function	b0	b1	b2
AND	-15	10	10
OR	-4	10	10

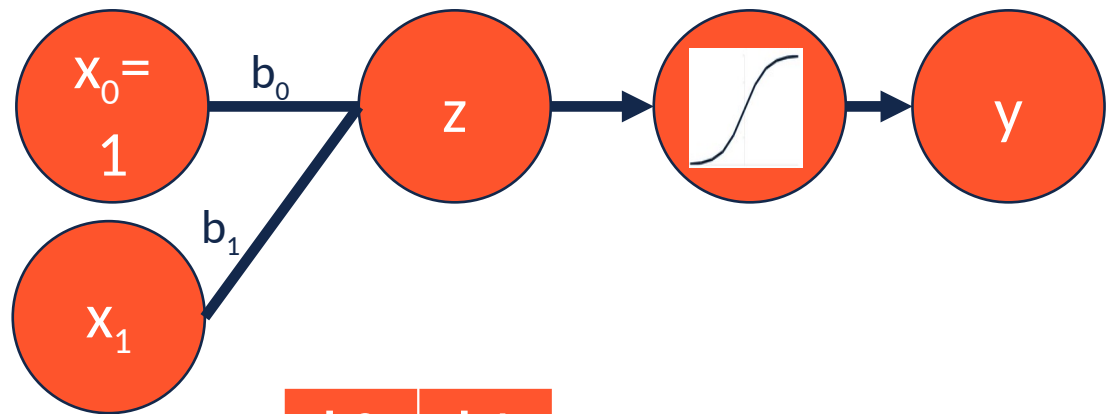
Takeaway: AND vs. OR

- For both functions, both inputs need to have the same positive weights
- But the two functions need to have different bias values
- Bias = neuron's threshold or baseline activity



Logical Computation in Neural Circuits

Logical NOT Neural Circuit



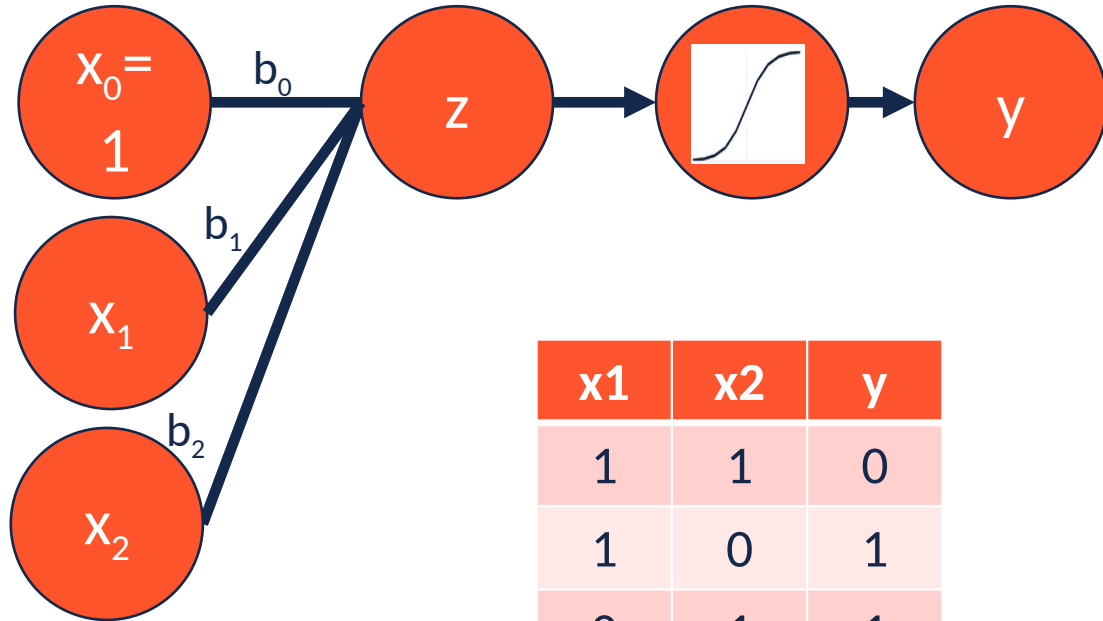
b_0	b_1
2	-5

x_0	x_1	y	o
1	1	0	.001
1	0	1	.999

Unit	b	x	$x*b$
x_0	5	1	5
x_1	-10	1	-10
z			-5
y			0.001

Unit	b	x	$x*b$
x_0	5	1	5
x_1	-10	0	0
z			5
y			0.999

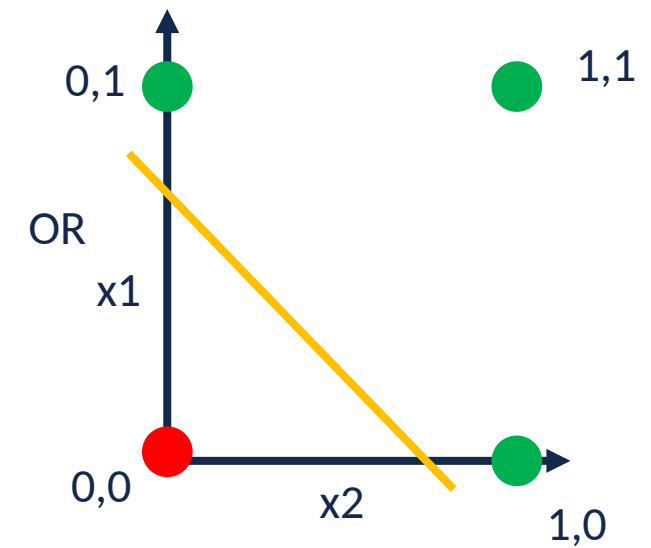
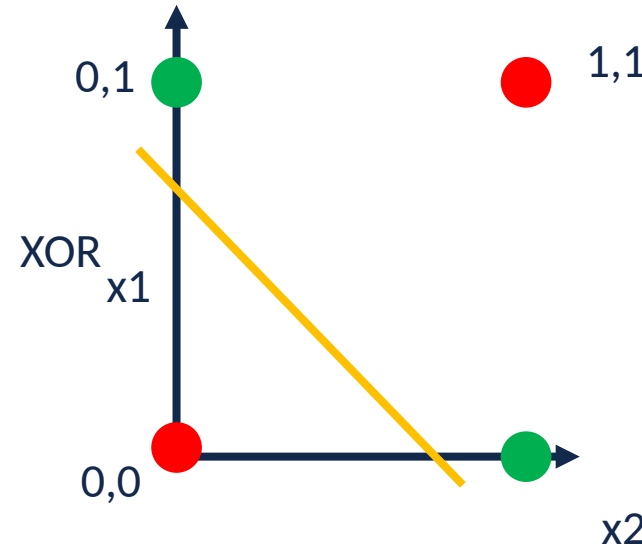
Logical Computation in Neural Circuits



x_1	x_2	y
1	1	0
1	0	1
0	1	1
0	0	0

Logical XOR

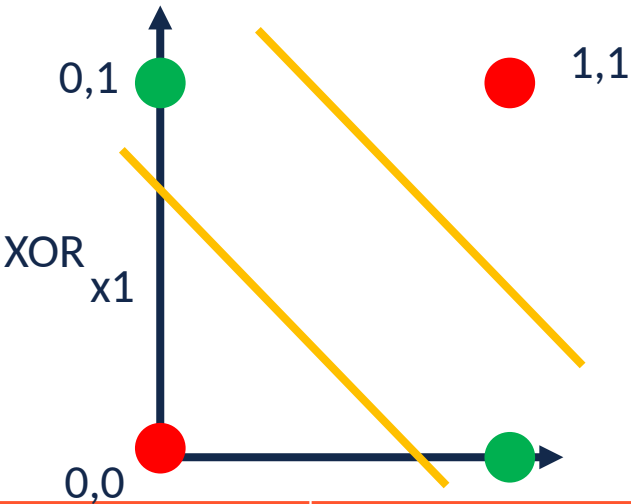
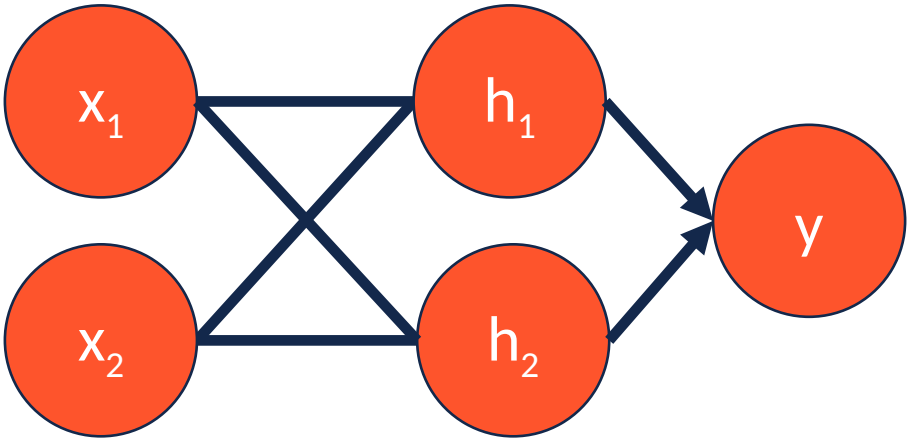
- XOR = output 1 if inputs are different
- Not linearly separable (can't do with 1 neuron)
- Solution: combine AND, OR, NOT



Logical Computation in Neural Circuits

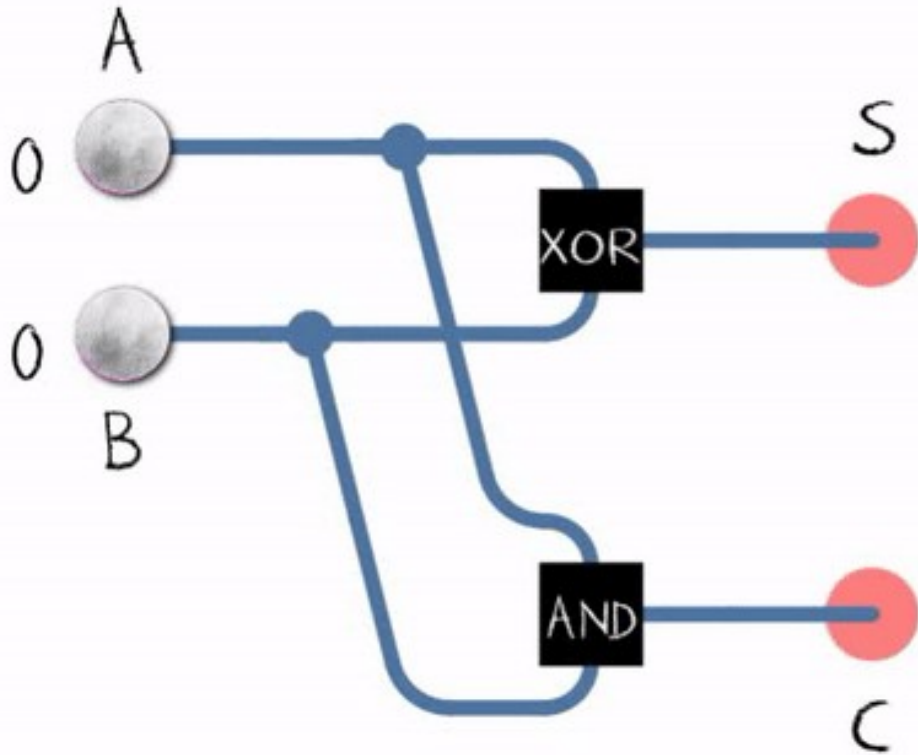
x1	x2	y
1	1	0
1	0	1
0	1	1
0	0	0

$$\text{XOR}(x_1, x_2) = \text{OR}(x_1, x_2) \text{ AND NOT}(\text{AND}(x_1, x_2))$$



x1	x2	h1=OR(x1,x2)	AND(x1,x2)	h2=NOT(AND(x1,x2))	y=AND(H1,H2)
1	1	0	0	1	0
1	0	1	0	1	1
0	1	1	0	1	1
0	0	1	1	0	0

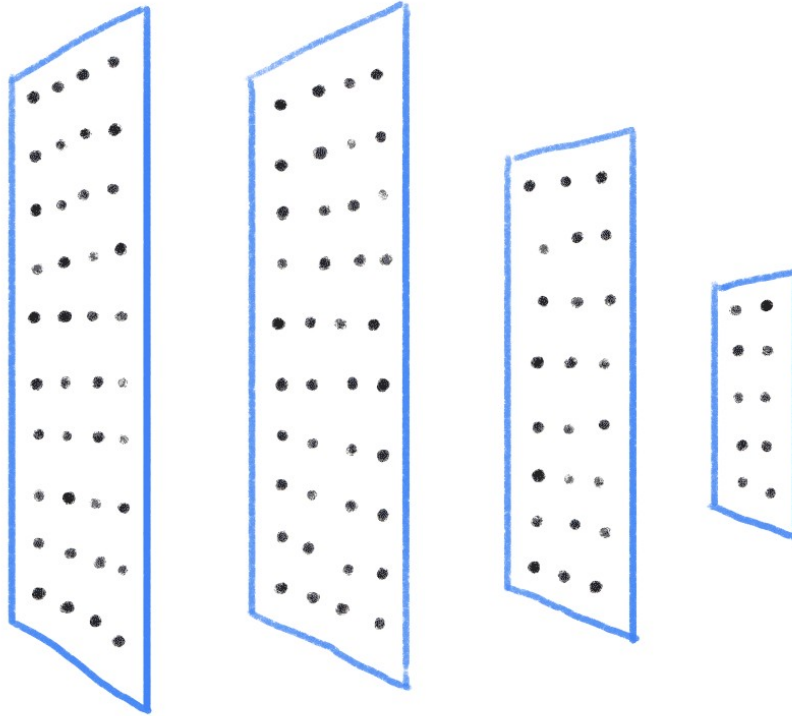
Logical Computation in Neural Circuits



Scaling Up: Adder Network

- Logic gates can be chained to do arithmetic
- Example: binary “full-adder” circuit
- Circuits can compute sums, carry bits

Logical Computation in Neural Circuits



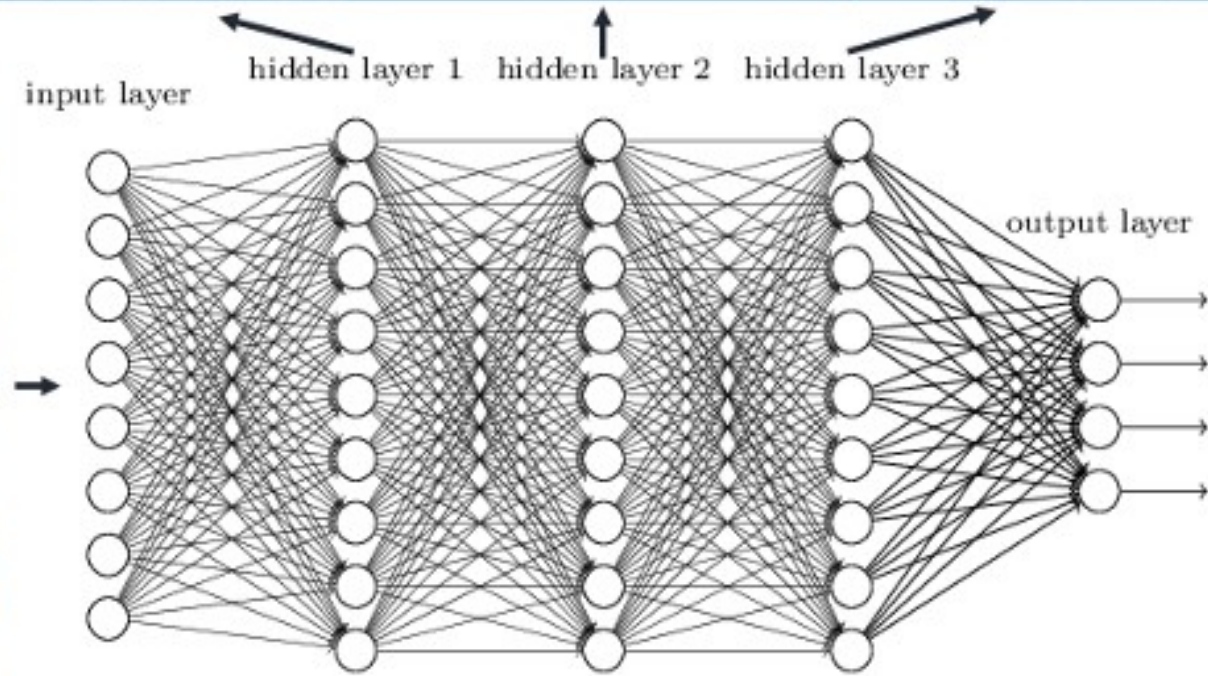
OUTPUT

Scaling Up: Visual Classifier

- Networks of neurons can classify patterns
- Example: distinguishing “cats” vs. “dogs”
- Simple network can learn decision boundary

Logical Computation in Neural Circuits

Deep neural networks learn hierarchical feature representations



Logical Computation in Neural Circuits

Recap & Overview

Lecture 1: Circuits as building blocks

Lecture 2: Circuits adapt & learn

Today: Circuits as computation

